



บทที่ 3

คำสั่งทำงานซ้ำ ขอบเขต การแปลงชนิดข้อมูล

บรรยายโดย ผศ.ดร.ธราวิเชษฐ์ ธิติจรูญโรจน์

หัวข้อ

01

คำสั่งทำงานซ้ำ

02

ขอบเขต

03

การแปลงชนิดข้อมูล

หัวข้อ

01

คำสั่งทำงานซ้ำ

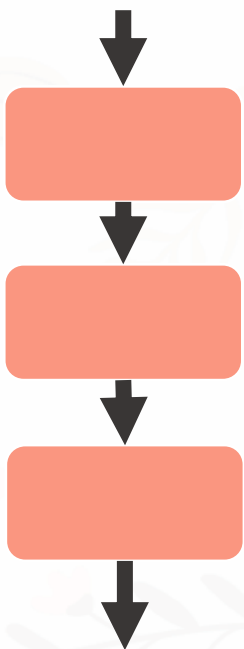
02

ขอบเขต

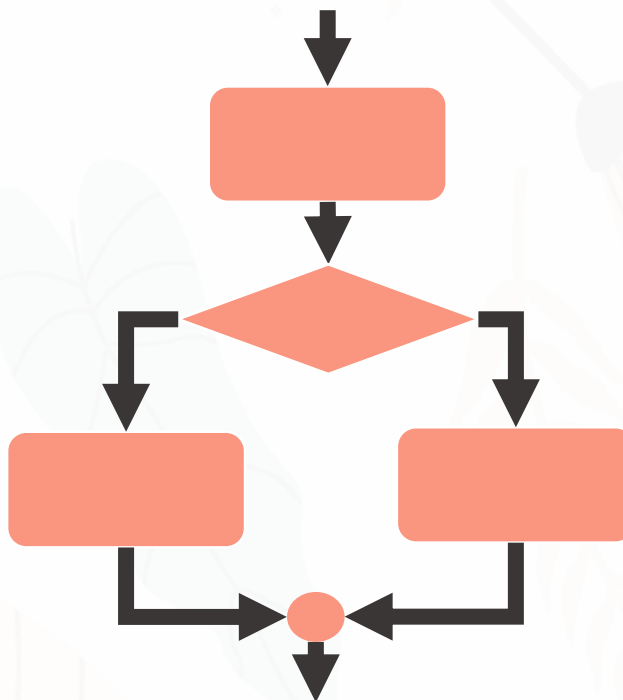
03

การแปลงชนิดข้อมูล

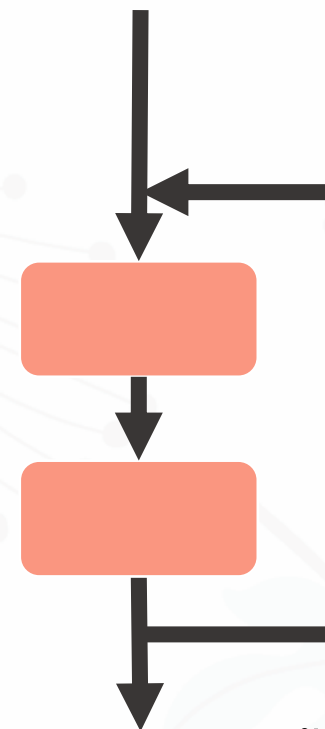
โครงสร้างการเขียนโปรแกรม



การทำงานแบบเรียงลำดับ
(Sequence)

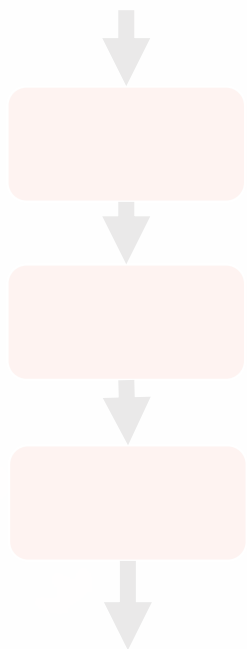


การทำงานแบบมีเงื่อนไข
(Selection)

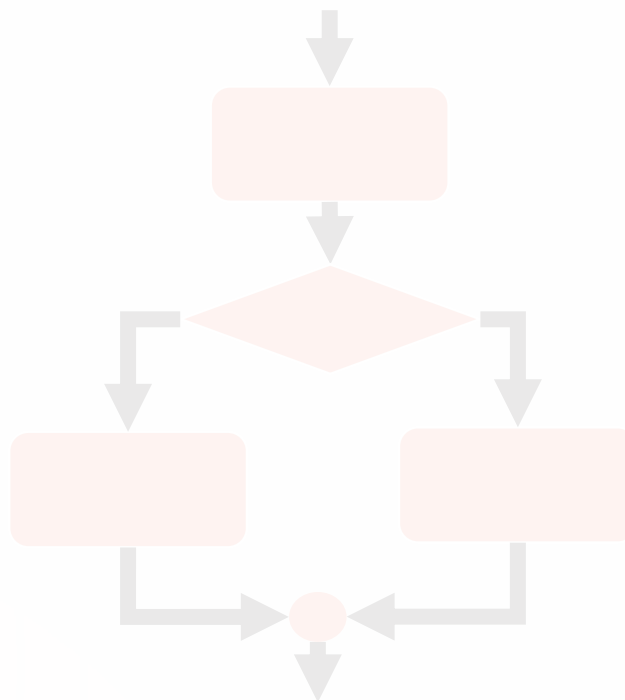


การทำงานแบบทำซ้ำ
(Iteration)

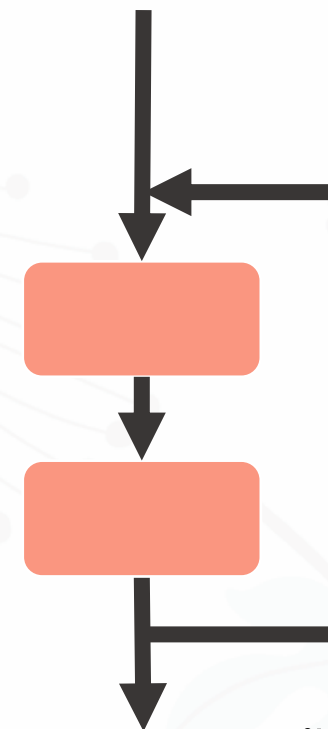
โครงสร้างการเขียนโปรแกรม



การทำงานแบบเรียงลำดับ
(Sequence)

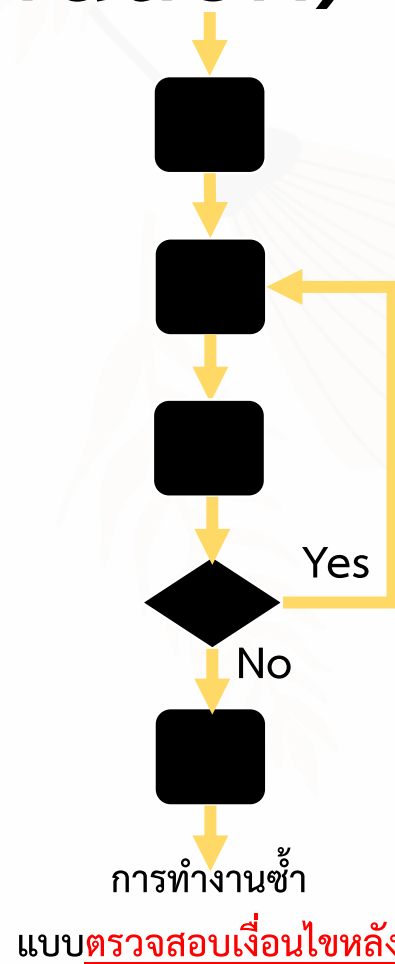
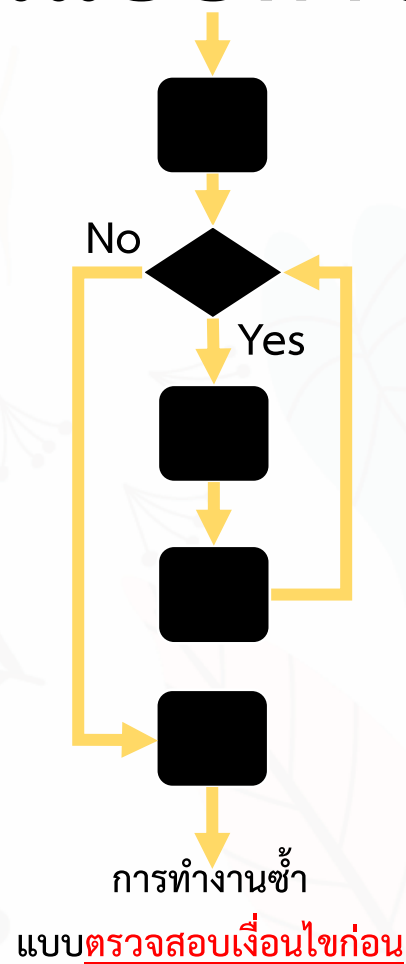


การทำงานแบบมีเงื่อนไข
(Selection)

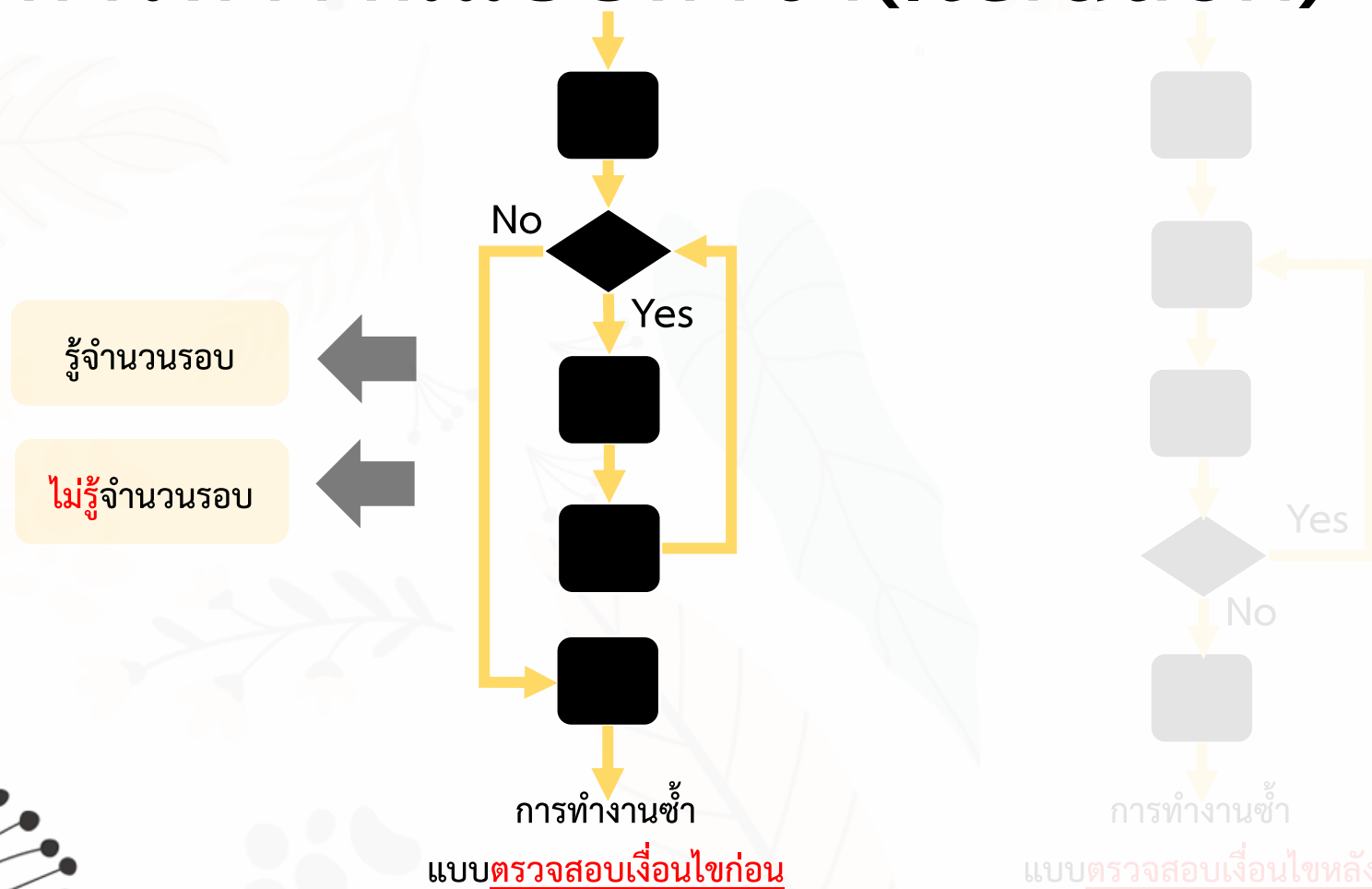


การทำงานแบบทำซ้ำ
(Iteration)

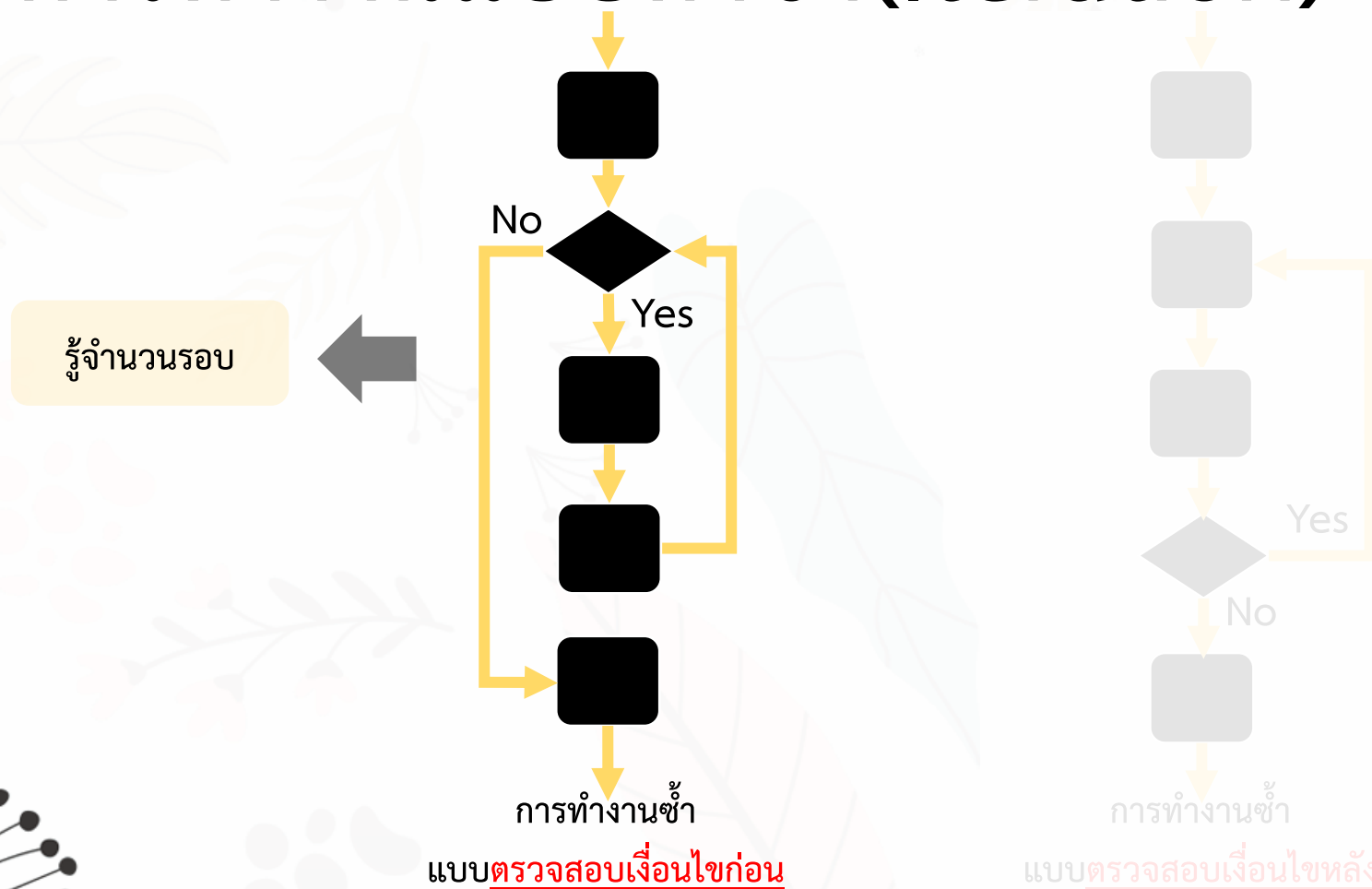
การทำงานแบบทำซ้ำ(Iteration)



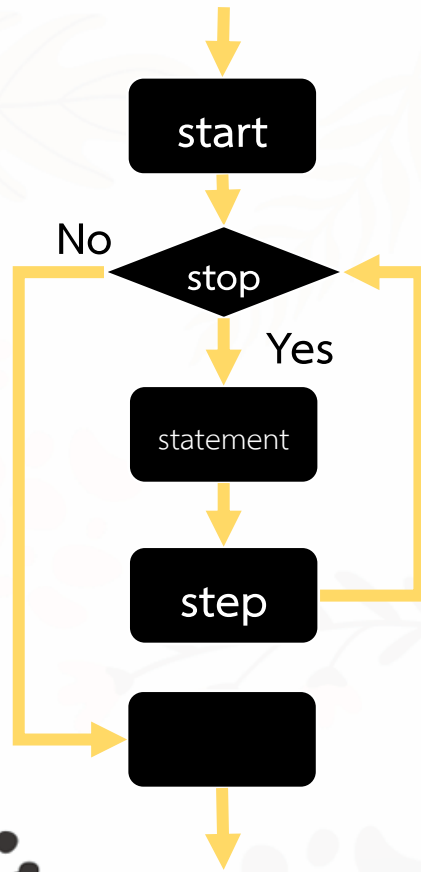
การทำงานแบบทำซ้ำ(Iteration)



การทำงานแบบทำซ้ำ(Iteration)



คำสั่ง for-loop



เป็นคำสั่งควบคุมการทำงานซ้ำตามเงื่อนไขที่กำหนด โครงสร้างพื้นฐานจะประกอบไปด้วย 3 ส่วน ได้แก่

```
for ( start; stop; step) {  
    statement  
}
```

การเริ่มต้น (initialization or start) คือ นิพจน์จะทำงานเพียงครั้งเดียวตั้งแต่จุดเริ่มต้นของ for-loop มักใช้เพื่อกำหนดค่าเริ่มต้นของตัวแปร loop

เงื่อนไข (condition or stop) คือ นิพจน์ที่ได้รับการประมวลผลก่อนการวนซ้ำแต่ละครั้ง หากเงื่อนไขนี้ยังเป็น true แต่ถ้าเงื่อนไขเป็น false การวนซ้ำจะสิ้นสุดลง

อัปเดต (update or step) คือ นิพจน์ที่จะประมวลผลหลังจากการวนซ้ำแต่ละครั้ง มักใช้เพื่ออัปเดตตัวแปร loop

คำสั่ง for-loop

- ตัวอย่างการทำงานซ้ำแบบรู้จำนวนรอบและ*ใช้ประโยชน์จาก running number*

```
for (int i = 1; i < 13; i++) {  
    System.out.println("2 x " + i + " = " + (2*i) );  
}
```

- ตัวอย่างการทำงานซ้ำแบบรู้จำนวนรอบและ*ไม่ใช้ประโยชน์จาก running number*

```
double balance = 100;  
for (int i = -5; i < 5; i++) {  
    balance *= 1.05;  
}  
System.out.println(balance);
```

คำสั่ง for-loop

โดยทั่วไปแล้วเราจะกำหนดนิพจน์ (1) การเริ่มต้น (2) เงื่อนไข และ (3) อัปเดต ของ For-loop

นิพจน์การเริ่มต้นและอัปเดตสามารถมีได้มากกว่า 1 แต่มีเงื่อนไขว่า (1) ในแต่ละนิพจน์จะต้องคั่นค่าเป็น void และ (2) แต่ละนิพจน์จะเชื่อมด้วยเครื่องหมาย “,” (comma)”

```
for (int i = 0; i < 5; i++){  
    System.out.println(i);  
}
```

นิพจน์เงื่อนไขสามารถมีได้เพียง 1 เงื่อนไขแต่สามารถสร้าง complex condition ที่ประกอบจากเงื่อนไขย่อยได้

คำสั่ง for-loop

ตัวอย่างการใช้ for-loop แบบนิพจน์การเริ่มต้นและอัปเดตสามารถมีได้มากกว่า 1 และนิพจน์เงื่อนไขเป็นแบบ Complex Condition ดังนี้

```
for (int i=0, j=0; i<4 & j < 10; i++, j+=2) {  
  
    statement  
  
}
```

คำสั่ง for-loop

นอกจากนี้ ขอบเขตของตัวแปรของ loop ที่ประกาศและกำหนดค่าในคำสั่ง for-loop จะใช้ได้เฉพาะภายในคำสั่ง for-loop เท่านั้น

```
public class Main{  
    public static void main(String args[]) {  
        for (int i=1; i<10; i++) {  
            System.out.print(i+" ");  
        }  
        System.out.println("i = "+i); //illegal  
    }  
}
```


คำสั่ง for-loop

ตัวอย่างการใช้ for-loop แบบ Loop Infinite ดังนี้

```
for (;;) {  
    // This will run forever  
}
```

แบบฝึกหัดคำสั่ง for-loop

```
public class Main {  
    public static void main(String[] args) {  
        for ( int x = __ (A) __; __ (B) __; __ (C) __ ) {  
            ....  
        }  
    }  
}
```

จงเขียนผลลัพธ์ลงในตารางต่อไปนี้

ค่า x ที่คาดหวัง	(A)	(B)	(C)
1, 4, 7, 10, 13, 16, 19			
-100, -90, -80, -70, -60, -50, -40			
-10, -5, 0, 5, 10, 15, 20			

แบบฝึกหัดคำสั่ง for-loop

```
public class Main {  
    public static void main(String[] args) {  
        for ( int x = 3 ; x < 7 ; x++ ){  
            System.out.print(x+2);  
        }  
    }  
}
```

จงเขียนผลลัพธ์ลงในตารางต่อไปนี้

รอบที่	ค่าตัวแปร x	System.out.print(x+2);

แบบฝึกหัดคำสั่ง for-loop

```
public class Main {  
    public static void main(String[] args) {  
        for ( int x = 10 ; x > 5 ; x-=2 ){  
            System.out.print("a-" + x);  
        }  
    }  
}
```

จงเขียนผลลัพธ์ลงในตารางต่อไปนี้

รอบที่	ค่าตัวแปร x	System.out.print("a-" + x);

คำสั่ง for-loop แบบ Nested Structure

นักศึกษาสามารถเขียนคำสั่งโครงสร้างควบคุมใด ๆ ซ้อนอยู่ภายในได้ ซึ่งโครงสร้างควบคุมภายในและภายนอกไม่จำเป็นต้องเป็นคำสั่งชนิดเดียวกัน ตัวอย่างการเขียนโครงสร้างทำซ้ำแบบซ้อน (for อยู่ใน for)

```
public class Main{  
    public static void main(String args[]) {  
        for (int i=1; i<=3; i++) {  
            for (int j=1; j<=5; j++) {  
                System.out.print('*') ;  
            }  
            System.out.println() ;  
        }  
    }  
}
```

ผลลัพธ์

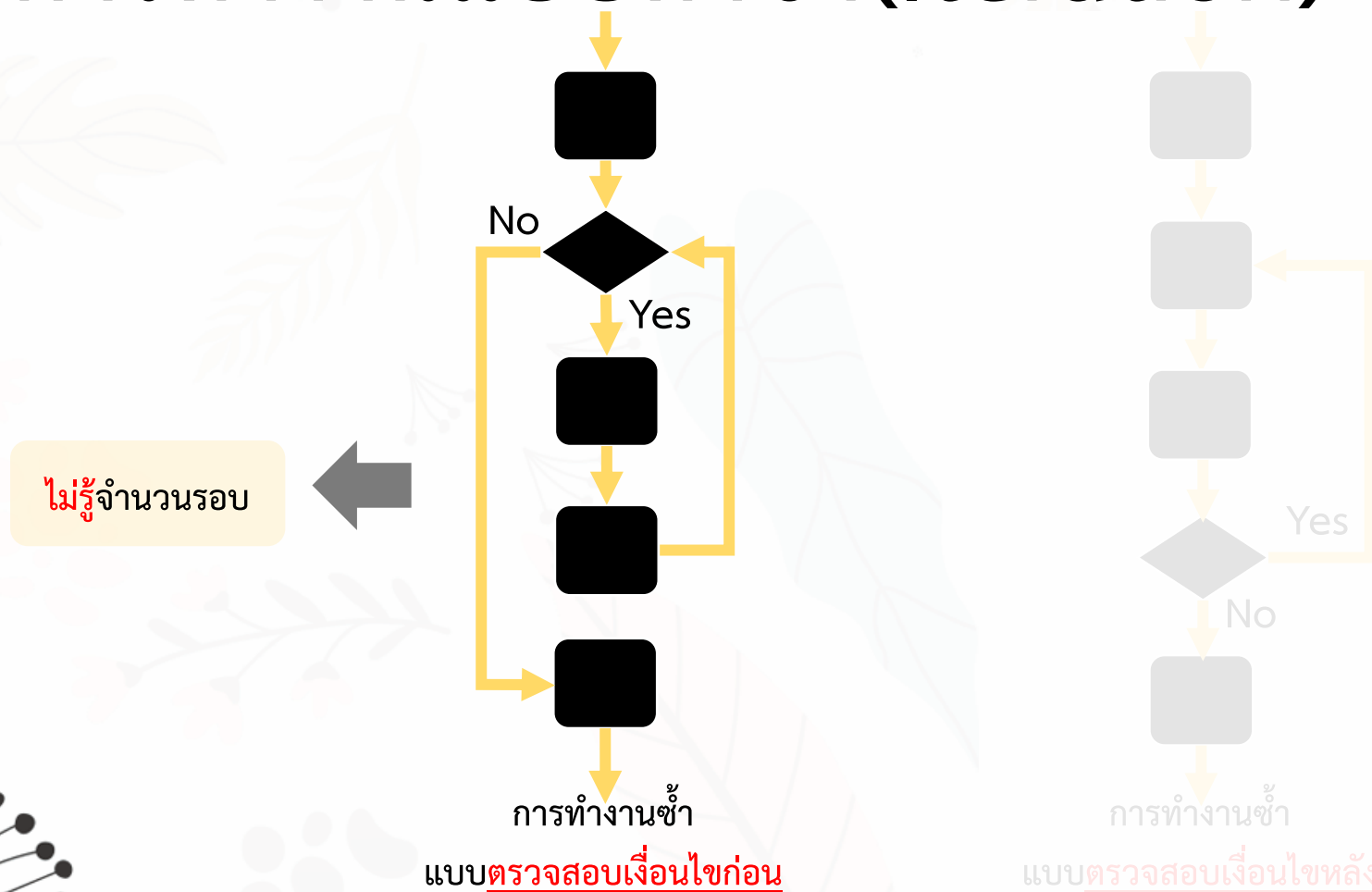
โดยที่ for ที่อยู่ภายใน จะพิมพ์เครื่องหมาย '*' เท่ากับจำนวนคอลัมน์ (column) และ for ที่อยู่ภายนอก จะทำคำสั่ง for ภายในเท่ากับจำนวนแถว (row)

คำสั่ง for-loop แบบ Nested Structure

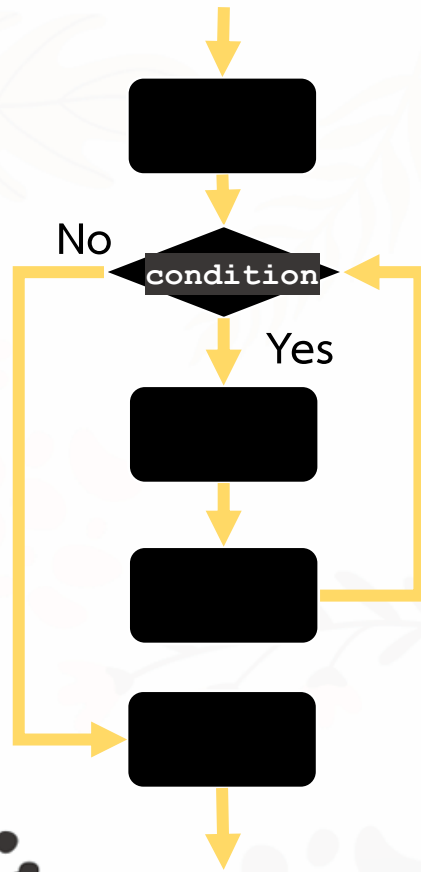
นักศึกษาสามารถเขียนคำสั่งโครงสร้างควบคุมใด ๆ ซ้อนอยู่ภายในได้ ซึ่งโครงสร้างควบคุมภายในและภายนอกไม่จำเป็นต้องเป็นคำสั่งชนิดเดียวกัน ตัวอย่างการเขียนโครงสร้างทำซ้ำแบบซ้อน (for อยู่ใน for)

```
public class Main {  
    public static void main(String[] args) {  
        for (int x = 1; x <=4; x++) {  
            if (x%2 == 0) {  
                System.out.println(x + " is even.");  
            } else {  
                System.out.println(x + " is odd.");  
            }  
        }  
    }  
}
```

การทำงานแบบทำซ้ำ(Iteration)



คำสั่ง while



เป็นคำสั่งควบคุมการทำงานซ้ำตามเงื่อนไขที่กำหนด ตราบใดที่เงื่อนไขที่ระบุเป็น true โครงสร้างพื้นฐานจะประกอบไปด้วยรายละเอียดดังนี้

```
while (condition) {  
    statement  
}
```

เงื่อนไข (condition) คือ นิพจน์ที่จะคืนค่าเป็น true หรือ false ซึ่งจะได้รับการตรวจสอบทุก ครั้งก่อนการวนซ้ำในแต่ละครั้ง หากเงื่อนไขประเมินผลแล้วเป็น true โค้ดส่วน statement ของ while-loop จะได้รับการประมวลผล หากเป็นเท็จ การวนซ้ำจะสิ้นสุดลง

คำสั่ง while

ตัวอย่างการใช้ while

```
public class Main{  
    public static void main(String args[]) {  
        Scanner sc = new Scanner(System.in);  
        int money = sc.nextInt();  
        while (money <= 10000) {  
            money += sc.nextInt();  
        }  
    }  
}
```

ความแตกต่างระหว่าง for และ while

การเขียนคำสั่งควบคุมการทำงานซ้ำ (Loop) สำหรับกรณีที่นักศึกษาไม่ทราบจำนวนรอบในการทำงานซ้ำ แต่ทราบเงื่อนไขในการหยุดการทำงาน ในทางปฏิบัติแล้วสามารถใช้งานได้ทั้ง for-loop และ while ดังตัวอย่างต่อไปนี้

```
public class Main {  
    public static void main(String[] args ) {  
        Scanner sc = new Scanner(System.in);  
        int money = sc.nextInt();  
        while (money <= 10000) {  
            money += sc.nextInt();  
        }  
    }  
}
```

1

```
public class Main {  
    public static void main( String[] args ) {  
        Scanner sc = new Scanner(System.in);  
        int money = sc.nextInt();  
        for (; money <= 10000;) {  
            money += sc.nextInt();  
        }  
    }  
}
```

2

จากตัวอย่างโปรแกรมข้างต้นจะพบว่าโปรแกรมที่ (1) และ (2) สามารถทำงานและได้ผลลัพธ์ที่เหมือนกัน แต่ถ้าพิจารณาถึงความเหมาะสมจะพบว่าคำสั่งควบคุมการทำงานซ้ำในกรณีไม่ทราบจำนวนรอบในการทำงานซ้ำ แต่ทราบเงื่อนไขในการหยุดการทำงาน นักศึกษาควรใช้ while มากกว่า for-loop

ความแตกต่างระหว่าง for และ while

การเขียนคำสั่งควบคุมการทำงานซ้ำ (Loop) สำหรับกรณีที่นักศึกษา ทราบจำนวนรอบในการทำงานซ้ำที่แน่นอน ในทางปฏิบัติ นักศึกษาสามารถใช้งานได้ทั้ง for-loop และ while ดังตัวอย่างต่อไปนี้

```
public class Main {  
    public static void main(String[] args ) {  
        int num = 1;  
        while (num <= 10) {  
            System.out.println(num);  
            num++;  
        }  
    }  
}
```

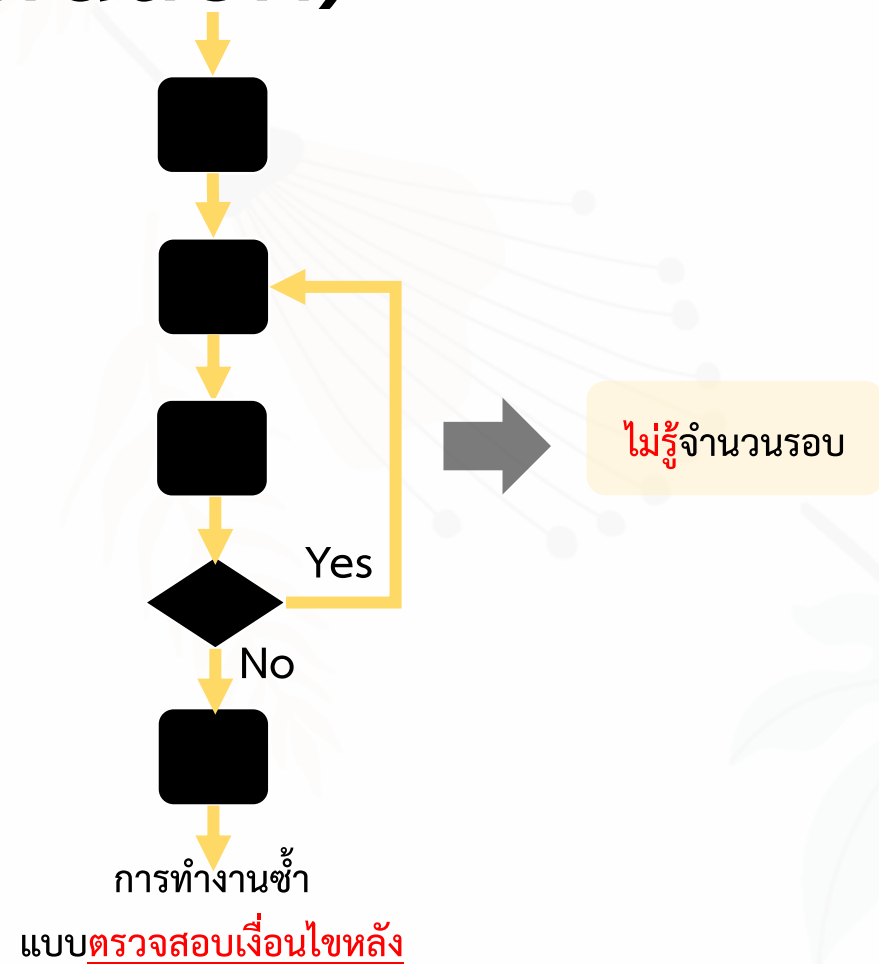
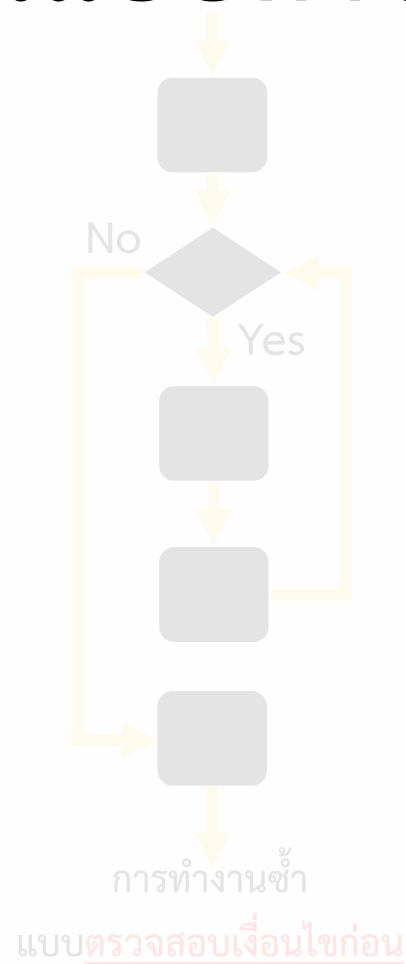
1

```
public class Main {  
    public static void main( String[] args ) {  
        for (int n = 1; n <= 10; n++) {  
            System.out.println(n);  
        }  
    }  
}
```

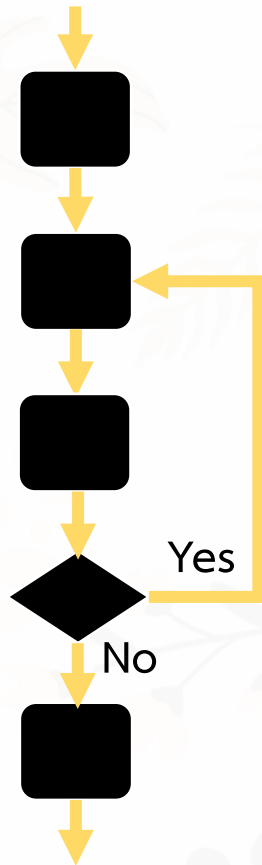
2

จากตัวอย่างโปรแกรมข้างต้นจะพบว่าโปรแกรมที่ (1) และ (2) สามารถทำงานและได้ผลลัพธ์ที่เหมือนกัน แต่ถ้าพิจารณาถึงความเหมาะสมจะพบว่าคำสั่งควบคุมการทำงานซ้ำที่ทราบจำนวนรอบในการทำงานซ้ำที่แน่นอน นักศึกษาควรใช้ for-loop มากกว่า while

การทำงานแบบทำซ้ำ(Iteration)



คำสั่ง do-while



คำสั่ง do- while loop ในภาษา Java เป็นรูปแบบหนึ่งของ while-loop เป็นคำสั่งควบคุมการทำงานซ้ำ เรียกใช้บล็อกโค้ดซ้ำๆ อย่างน้อยหนึ่งครั้ง จากนั้น จึงดำเนินการต่อไปตามเงื่อนไขที่ระบุเป็น true โดยมีโครงสร้างพื้นฐานดังนี้

```
do {  
    statement  
} while ( condition );
```

คุณสมบัติที่สำคัญของ do- while loop

- **รับประกันการดำเนินการ** กล่าวคือ ไม่เหมือนกับ while loop ที่มีการตรวจสอบเงื่อนไขก่อนที่ตัว loop จะถูกดำเนินการ แต่ do- While loop จะทำให้แน่ใจได้ว่าตัว loop จะถูกดำเนินการอย่างน้อยหนึ่งครั้ง ก่อนที่จะมีการตรวจสอบเงื่อนไข
- **เงื่อนไข** กล่าวคือ หลังจากดำเนินการกับเนื้อหาของลูปแล้ว เงื่อนไขนี้จะได้รับการประมวลผล หากเงื่อนไขประเมินเป็นจริง นิพจน์ภายในของ loop จะถูกดำเนินการอีกครั้ง หากประเมินเป็น false การวนซ้ำจะสิ้นสุดลง

คำสั่ง do-while

ตัวอย่างการใช้ คำสั่ง do-while

```
public class Main{  
    public static void main(String args[]) {  
        Scanner sc = new Scanner(System.in);  
        int money = 0;  
        do{  
            money += sc.nextInt();  
        }while (money <= 10000);  
    }  
}
```

ความแตกต่างระหว่าง while และ do-while

ความแตกต่างหลักระหว่าง while loop และ do-While loop อยู่ที่ลำดับการประมวลผลเงื่อนไขของ loop ที่สัมพันธ์กับการประมวลผลนิพจน์ภายใน loop

```
public class Main {  
    public static void main(String[] args ) {  
        Scanner sc = new Scanner(System.in);  
        int money = sc.nextInt();  
        while (money <= 10000) {  
            money += sc.nextInt();  
        }  
    }  
}
```

1

```
public class Main {  
    public static void main( String[] args ) {  
        Scanner sc = new Scanner(System.in);  
        int money = 0;  
        do {  
            money += sc.nextInt();  
        } while (money <= 10000);  
    }  
}
```

2

while-loop มีการตรวจสอบเงื่อนไขก่อนที่ตัวลูปจะถูกดำเนินการ หากเงื่อนไขเป็น false ตั้งแต่ต้น นิพจน์ภายใน loop อาจไม่ดำเนินการเลย ในขณะที่ do-while จะประมวลผลนิพจน์ภายใน loop ก่อนที่จะตรวจสอบเงื่อนไข สิ่งนี้รับประกันได้ว่านิพจน์ภายใน loop จะดำเนินการอย่างน้อยหนึ่งครั้ง โดยไม่คำนึงถึงเงื่อนไข

คำสั่ง do-while

ตัวอย่างการใช้ คำสั่ง do-while

```
public class Main{  
    public static void main(String args[]) {  
        Scanner sc = new Scanner(System.in);  
        String password;  
        do {  
            password = sc.nextLine();  
        } while (password.equals("1234"));  
    }  
}
```

คำสั่ง break และ continue

คำสั่ง break และคำสั่ง continue คือ คำสั่งควบคุมการทำงานที่นิยมใช้งานในการะบวนการวนซ้ำให้มีความยืดหยุ่นมากขึ้น โดยการเปลี่ยน Flow การทำงานของโปรแกรมตาม Loop ปกติ ซึ่งคำสั่ง break และ continue มีความแตกต่างในการใช้งานดังนี้

```
while (condition){  
    statement  
    statement  
    break  
    statement  
    statement  
}
```



1

```
while (condition){  
    statement  
    statement  
    continue  
    statement  
    statement  
}
```



2

คำสั่ง break

มีรายละเอียดดังนี้

หัวข้อ	เนื้อหา
วัตถุประสงค์	• สำหรับคำสั่ง Loop ใช้เพื่อออกจากคำสั่งวนรอบปัจจุบัน • สำหรับคำสั่ง Switch ใช้เพื่อหยุดการทำงานภายในคำสั่ง switch ก่อนเวลาอันควร
การใช้งาน	• สำหรับคำสั่ง Loop จะยุติการวนซ้ำของ Loop ดังกล่าว • สำหรับคำสั่ง Switch จะออกจากคำสั่ง switch

ตัวอย่างโปรแกรม

```
for (int i = 0; i < 5; i++) {  
    System.out.print("<");  
    if (i == 2)  
        break;  
    System.out.print(i + ">");  
}
```

คำสั่ง continue

มีรายละเอียดดังนี้

หัวข้อ	เนื้อหา
วัตถุประสงค์	คำสั่ง Continue จะข้ามการวนซ้ำในรอบปัจจุบันของ Loop และข้ามไปยังการวนซ้ำครั้งถัดไป
การใช้งาน	ใช้ใน Loop เพื่อข้ามส่วนที่เหลือของนิพจน์ภายในการวนซ้ำของรอบปัจจุบัน และดำเนินการตรวจสอบเงื่อนไขของลูปหรือการวนซ้ำครั้งถัดไป

ตัวอย่างโปรแกรม

```
for (int i = 0; i < 5; i++) {  
    System.out.print("<");  
    if (i == 2)  
        continue;  
    System.out.print(i + ">");  
}
```

คำสั่ง label with break และ continue

คำสั่ง label มักจะใช้งานร่วมกับคำสั่ง Loop และคำสั่ง break หรือ continue ทำให้ควบคุม Flow ของโปรแกรมได้ละเอียดยิ่งขึ้น ซึ่งช่วยให้นักศึกษากำหนดได้ว่าคำสั่ง break หรือ continue จะส่งผลกระทบต่อ Loop ไດ สิ่งนี้มีประโยชน์อย่างยิ่งเมื่อต้องควบคุมการทำงานกับ Loop ที่ซ้อนกัน

ประเด็นสำคัญ

- คำสั่ง label เป็นวิธีควบคุม Loop ด้านนอก โดยใช้คำสั่ง break หรือ continue จากภายใน Loop ด้านใน
- คำสั่ง label จะไม่ใช่ในการเขียนโปรแกรมสมัยใหม่ เนื่องจากสามารถทำได้ อ่านยากขึ้น อย่างไรก็ตาม อาจมีประโยชน์ในบางสถานการณ์ที่มีการวนซ้ำแบบซ้อนที่ซับซ้อน
- คำสั่ง label จำเป็นอย่างยิ่งที่จะต้องใช้อย่างรอบคอบและให้แน่ใจว่าโค้ดยังคงสามารถอ่านและบำรุงรักษาได้

ตัวอย่างโปรแกรม

```
outerLoop: for (int i = 0; i < 5; i++) {  
    for (int j = 0; j < 5; j++) {  
        if (i * j > 6) {  
            System.out.println("Breaking from outerLoop");  
            break outerLoop;  
        }  
    }  
}
```

1

```
outerLoop: for (int i = 0; i < 5; i++) {  
    for (int j = 0; j < 5; j++) {  
        if (i * j > 6) {  
            System.out.println("continuing from outerLoop");  
            continue outerLoop;  
        }  
    }  
}
```

2

```

public class Main {
    public static void main(String[] args) {
        Scanner s = new Scanner(System.in);
        System.out.print("กรุณาใส่ความสูงของชั้นที่ 1 :");
        int t1 = s.nextInt();
        System.out.print("กรุณาใส่ความสูงของชั้นที่ 2 :");
        int t2 = s.nextInt();
        System.out.println("  #");
        System.out.println(" ##");
        System.out.println("###");

        for(int x = 1; x <= t1;x++){
            System.out.println("=A=");
        }

        System.out.println("###");
        System.out.println("###");
        System.out.println("###");

        for(int x = 1; x <= t2;x++){
            System.out.println("=B=");
        }

        System.out.println("###");
        System.out.println(" ##");
        System.out.println("  #");
    }
}

```

```

กรุณาใส่ความสูงของชั้นที่ 1 : 3
กรุณาใส่ความสูงของชั้นที่ 2 : 6
#
##
###
=A=
=A=
=A=
###
###
###
=B=
=B=
=B=
=B=
=B=
=B=
###
##
#
-

```

```

กรุณาใส่ความสูงของชั้นที่ 1 : 4
กรุณาใส่ความสูงของชั้นที่ 2 : 2
#
##
###
=A=
=A=
=A=
=A=
###
###
###
=B=
=B=
###
##
#

```

หัวข้อ

01

คำสั่งทำงานซ้ำ

02

ขอบเขต

03

การแปลงชนิดข้อมูล

ขอบเขต

การระบุขอบเขตในภาษาจาวาจะพิจารณาจาก 3 ตัวดำเนิน ได้แก่ semicolon ; วงเล็บ (...) และ วงเล็บปีกกา {...}

```
public class Main {  
    public static void main(String[] args) {  
  
        int a = 10;  
  
        System.out.println("line 1: "+a);  
  
    }  
}
```

ขอบเขต

ตัวอย่างที่ 1

```
public class Main {  
    public static int num = 10;  
    public static void main(String[] args) {  
        System.out.println("Num#1 in Main is "+num);  
        System.out.println("Num#2 in Main is "+num);  
    }  
}
```

Output - JavaApplication7 (run)

```
run:  
Num#1 in Main is 10  
Num#2 in Main is 10  
BUILD SUCCESSFUL (to
```

ตัวอย่างที่ 2

```
public class Main {  
    public static int num = 10;  
    public static void main(String[] args) {  
        System.out.println("Num#1 in Main is "+num);  
        int num = 9;  
        System.out.println("Num#2 in Main is "+num);  
    }  
}
```

Output - JavaApplication7 (run)

```
run:  
Num#1 in Main is 10  
Num#2 in Main is 9  
BUILD SUCCESSFUL (to
```

ขอบเขต

```
public class Main {  
    public static int num = 10;  
    public static void main(String[] args) {  
        System.out.println("Num#1 in Main is "+num);  
  
        int num = 9;  
        System.out.println("Num#2 in Main is "+num);  
  
        num += (9+3*7);  
        System.out.println("Num#3 in Main is "+num);  
    }  
}
```

ขอบเขต

```
public class Main {  
    public static void main(String[] args) {  
        int a = 1;  
        if ( a == 1 ){  
            int b = 20;  
            System.out.println("Var A is " + a);  
            System.out.println("Var B is " + b);  
        }  
        System.out.println("Var A is " + a);  
        System.out.println("Var B is " + b);  
    }  
}
```

หัวข้อ

01

คำสั่งทำงานซ้ำ

02

ขอบเขต

03

การแปลงชนิดข้อมูล

การแปลงชนิดข้อมูล

วิธีที่ 1 การแปลงผ่านเครื่องหมาย =

แปลงข้อมูลเป็นขนาดใหญ่ขึ้นเท่านั้นถ้าแปลงเป็นขนาดเล็กกว่าจะเกิดข้อผิดพลาดขณะคอมไพล์

วิธีที่ 2 การแปลงเมื่อเกิดการดำเนินการ

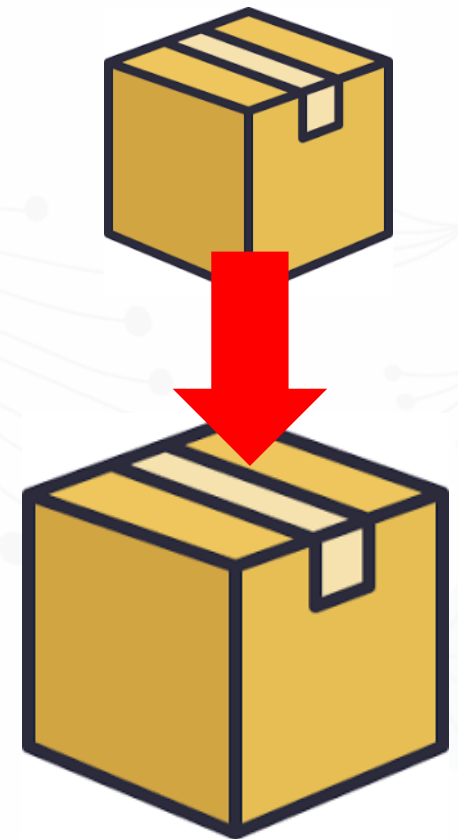
เมื่อ $x * y$ โดย x เป็น double และ y เป็น int ซึ่ง y จะถูกเปลี่ยนเป็น double ก่อนดำเนินการโดยอัตโนมัติ

วิธีที่ 3 การแปลงผ่านการทำ Casting

ลักษณะการใช้งาน เมื่อ x เป็น int และ y เป็น double เช่น `int x = (int) y;`

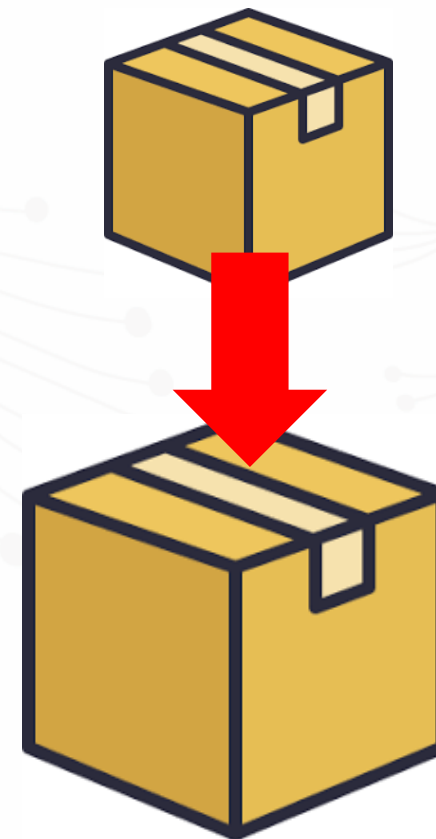
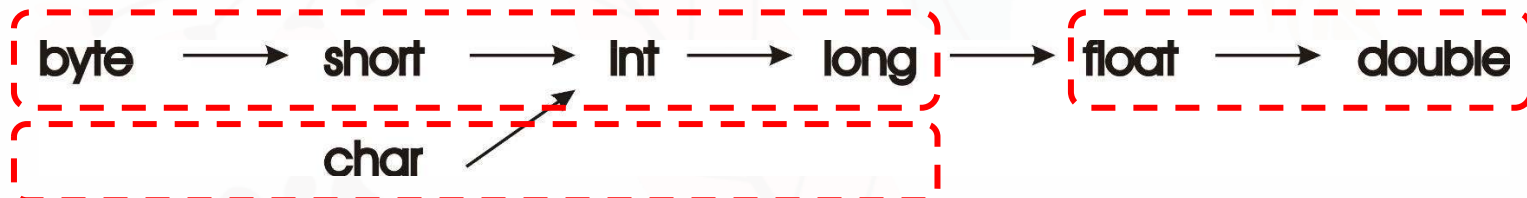
วิธีที่ 4 การใช้เมธอดจากคลาส Wrapper

ใช้เมธอดที่มาพร้อมคลาส เช่น `parseInt` หรือ `parseBoolean` เพื่อแปลงข้อมูลชนิดใด ๆ ก็ได้มาเป็นข้อมูลชนิด int หรือ boolean ตามลำดับ เป็นต้น



การแปลงชนิดข้อมูล

- ชนิดข้อมูล ตัวเลขจำนวนเต็ม สามารถแปลงให้เป็นชนิดข้อมูล ตัวเลขทศนิยม ได้
- ชนิดข้อมูล float สามารถแปลงให้เป็นชนิดข้อมูล double ได้
- ชนิดข้อมูล ตัวเลขจำนวนเต็ม มีขนาดเรียงกันจากน้อยไปมากดังนี้
- ชนิดข้อมูล char สามารถแปลงให้เป็นชนิดข้อมูล int ได้
- ชนิดข้อมูล boolean จะไม่มีความสัมพันธ์กับชนิดข้อมูลแบบพื้นฐานอื่นๆ



การแปลงชนิดข้อมูล

วิธีที่ 1 การแปลงผ่านเครื่องหมาย =

```
public class Main {  
    public static void main(String[] args) {  
        int a = 10;  
        double b = a;  
    }  
}
```

วิธีที่ 2 การแปลงเมื่อเกิดการดำเนินการ

```
public class Main {  
    public static void main(String[] args) {  
        int a = 10;  
        double b = 15.0;  
        System.out.println("line 1: " + (a+b));  
    }  
}
```

การแปลงชนิดข้อมูล

วิธีที่ 3 การแปลงผ่านการทำ Casting (ตัดข้อมูลออก)

```
public class Main {  
    public static void main(String[] args) {  
        double b = 15.7;  
        int a = (int) b;  
    }  
}
```

วิธีที่ 4 การใช้เมธอดจากคลาส Wrapper

```
public class App06 {  
    public static void main(String[] args) {  
        String str = "1506";  
        int num = Integer.parseInt(str);  
        System.out.println(num);  
    }  
}
```

SIMPLE TYPE	WRAPPER CLASSES
boolean	Boolean
char	Character
double	Double
float	Float
int	Integer
long	Long

ขอบเขต

```
public class Main {  
    public static void main(String[] args) {  
        // Convert int to String  
        int i = 10;  
        String str1 = String.valueOf(i);  
        String str2 = i + "";  
  
        // Convert String to int  
        int num1 = Integer.parseInt("200");  
  
        // Convert double to String  
        double num2 = 12.3;  
        String str3 = String.valueOf(num2);  
  
        // Convert String to double  
        double num3 = Double.parseDouble("23.6");  
    }  
}
```

ขอบเขต

```
public class Main {  
    public static void main(String args[]) {  
        byte  b1, b2, b3;  
        b1 = 2;  
        b2 = 4;  
        b3 = b1+b2;           //illegal  
    }  
}
```

คำสั่งกำหนดค่า b3 ไม่ถูกต้องเนื่องจาก b1+b2 จะให้ค่าข้อมูลที่มีชนิดข้อมูลเป็น int

ขอบเขต

Actual type	Computational type	Category
boolean	int	1
byte	int	1
char	int	1
short	int	1
int	int	1
float	float	1
reference	reference	1
returnAddress	returnAddress	1
long	long	2
double	double	2

เนื่องจากข้อจำกัดของขนาดของการเข้ารหัสแบบ one-byte operator code (opcode) ใน JVM เป็นข้อจำกัดที่ส่งผลต่อการออกแบบชุดคำสั่ง ทำให้ไม่สามารถออกแบบ opcode ให้สนับสนุนการดำเนินการสำหรับ Data Type ทั้งหมดได้ ดังนั้น จึงมีการแบ่งแยกชนิดข้อมูลตามความจำเป็นโดยแบ่งออกเป็น 2 กลุ่ม ได้แก่ unsupported และ supported data types สิ่งนี้ทำให้ตัวดำเนินการส่วนใหญ่ที่ดำเนินการกับชนิดข้อมูล byte, char และ short จะดำเนินการโดยอาศัยหลักการเดียวกับชนิดข้อมูลแบบ int

การแปลงชนิดข้อมูล

ภาษาจาวาจะปรับชนิดข้อมูลให้อัตโนมัติ ในกรณีต่อไปนี้

- กำหนดค่าชนิดข้อมูลที่เล็กกว่าให้กับตัวแปรชนิดข้อมูลที่ใหญ่กว่าอาทิเช่น

```
int i = 4;  
long l = i;
```

โดยที่ นิพจน์ i จะถูกปรับชนิดข้อมูลจาก int ให้เป็น long โดยอัตโนมัติ

- กำหนดค่าชนิดข้อมูลจำนวนเต็มให้กับจำนวนเลขทศนิยม อาทิเช่น

```
double x = 3;
```

โดยที่ นิพจน์ 3 จะถูกปรับชนิดข้อมูลจาก int ให้เป็น double โดยอัตโนมัติ ตัวอย่างที่ไม่ถูกต้อง

```
int amount = 123L;    //illegal  
float f = 4.0;        //illegal
```

ขอบเขต

```
public class Main {  
    public static void main(String args[]) {  
        byte b1 = 4, b2 = 3, b3;  
        b3 = (byte) (b1+b2);           // b3 = b1+b2;  
        float f1;  
        f1 = (float)3.2;               // f1 = 3.2;  
    }  
}
```

```
public class Main{  
    public static void main(String args[]) {  
        int i = 4;  
        long l = i;  
        float f1 = 4.2f;  
        f1 = i;  
        double x = f1;  
        f1 = 4.2;    //illegal  
    }  
}
```

การแปลงชนิดข้อมูล

วิธีที่ 1 การแปลงผ่านเครื่องหมาย =

แปลงข้อมูลเป็นขนาดใหญ่ขึ้นเท่านั้นถ้าแปลงเป็นขนาดเล็กกว่าจะเกิดข้อผิดพลาดขณะคอมไพล์

วิธีที่ 2 การแปลงเมื่อเกิดการดำเนินการ

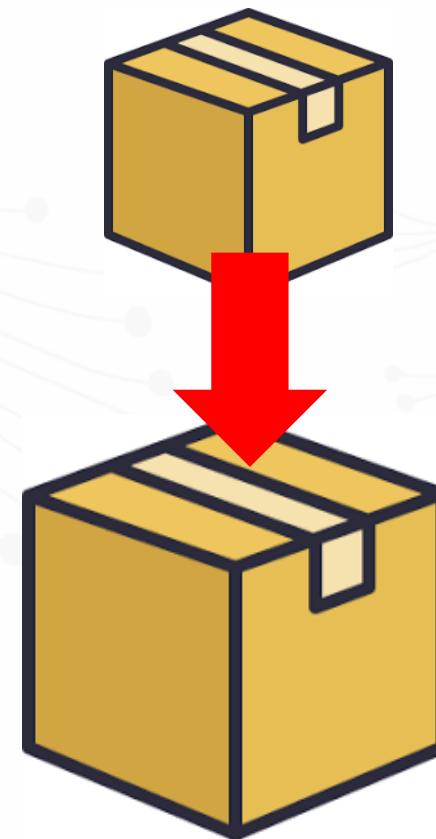
เมื่อ $x * y$ โดย x เป็น double และ y เป็น int ซึ่ง y จะถูกเปลี่ยนเป็น double ก่อนดำเนินการโดยอัตโนมัติ

วิธีที่ 3 การแปลงผ่านการทำ Casting

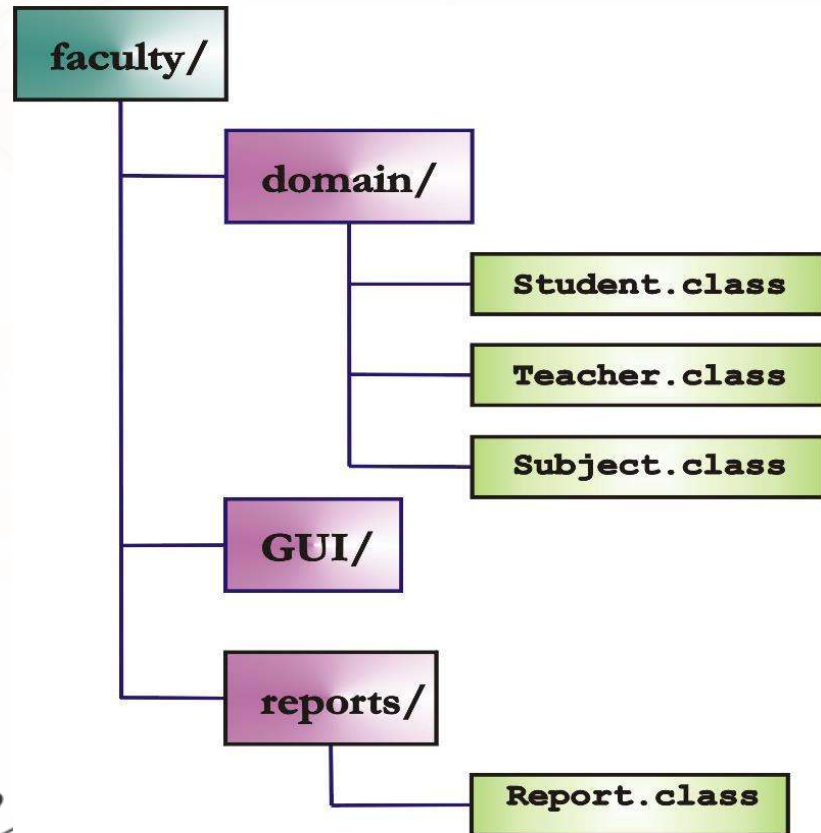
ลักษณะการใช้งาน เมื่อ x เป็น int และ y เป็น double เช่น `int x = (int) y;`

วิธีที่ 4 การใช้เมธอดจากคลาส Wrapper

ใช้เมธอดที่มาพร้อมคลาส เช่น `parseInt` หรือ `parseBoolean` เพื่อแปลงข้อมูลชนิดใด ๆ ก็ได้มาเป็นข้อมูลชนิด int หรือ boolean ตามลำดับ เป็นต้น



แพ็คเกจ (Package)



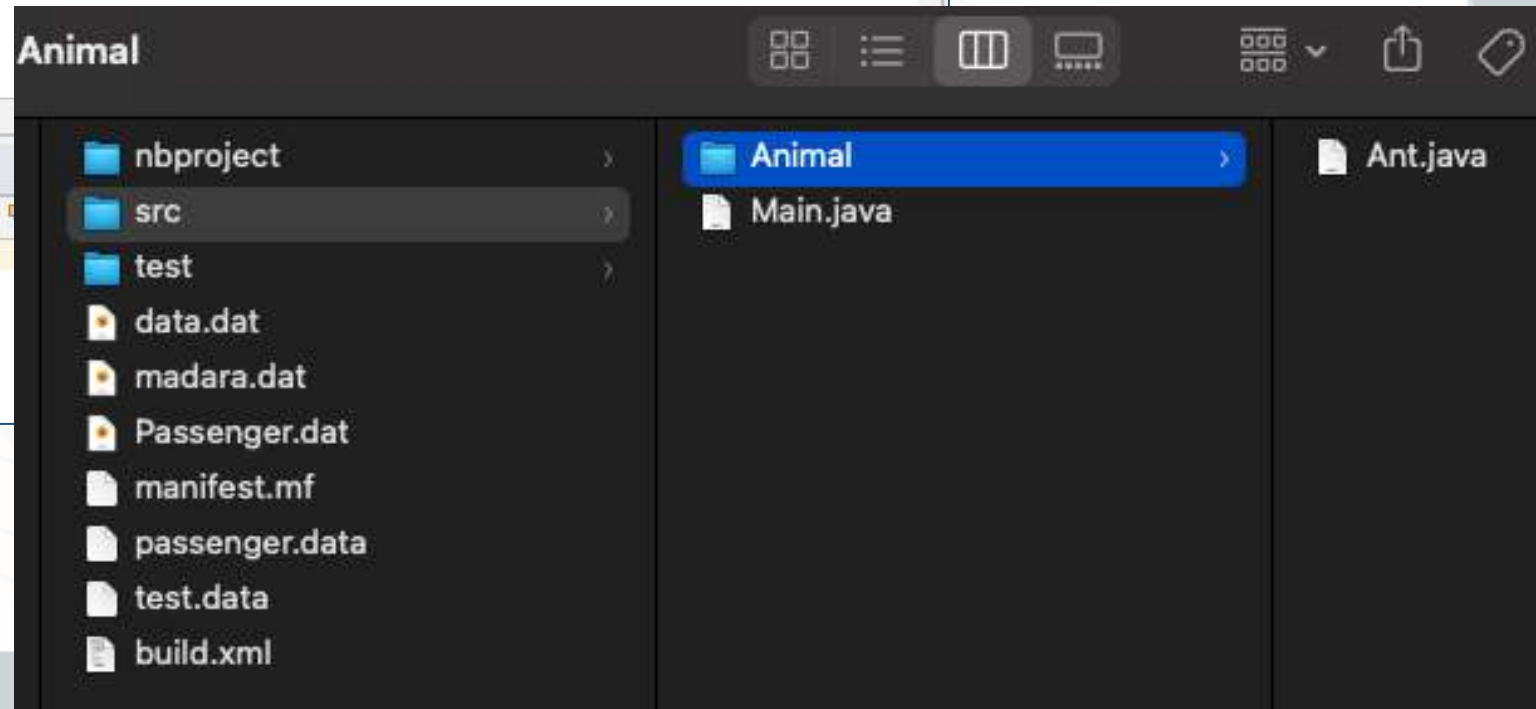
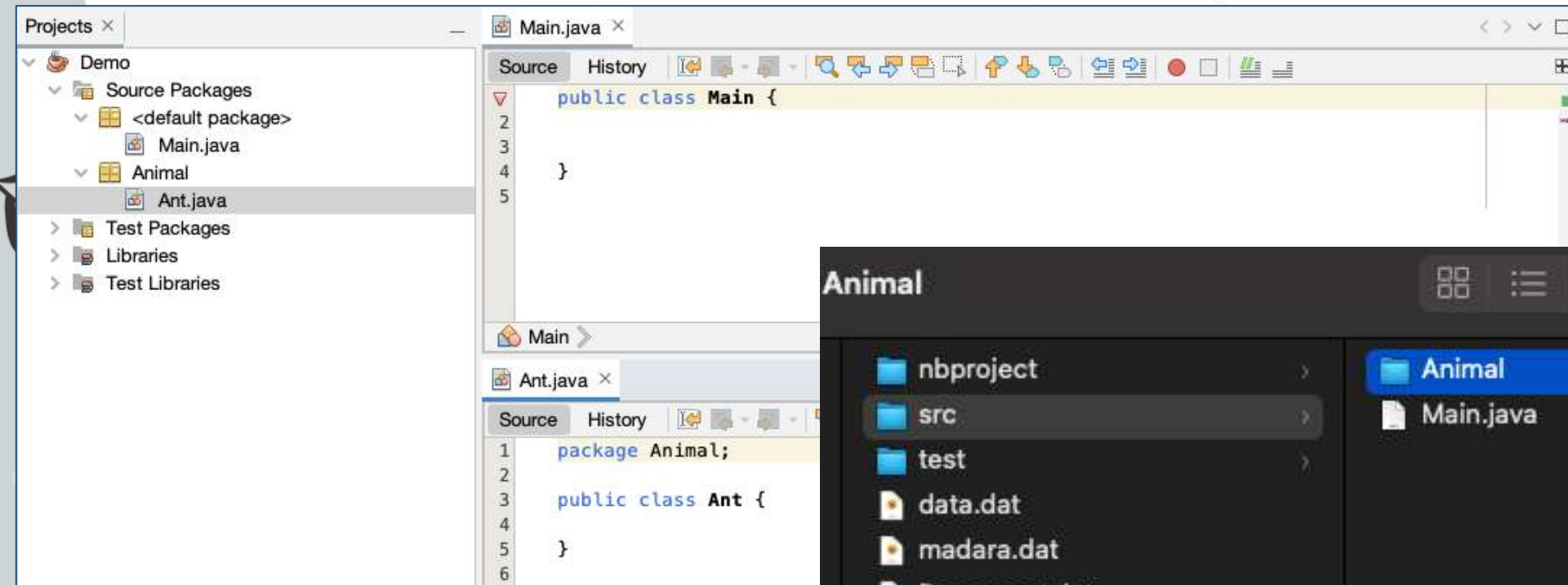
- ซอฟต์แวร์แพ็คเกจช่วยในการจัดการการพัฒนาโปรแกรมขนาดใหญ่
- ในโปรแกรมภาษาจาวา แพ็คเกจจะเป็นที่รวมของคลาสของภาษาจาวาหลาย ๆ คลาส
- โปรแกรมอาจแบ่งเป็นแพ็คเกจและแพ็คเกจย่อย (Subpackage)
- แพ็คเกจจะเก็บไว้ในไดเรกทอรี (Directory) ซึ่งจะเป็นชื่อของแพ็คเกจ

แพ็คเกจ (Package)

รูปแบบโปรแกรมภาษาจาวามีดังนี้

```
package_declaration  
import_declaration  
class_declaration
```


แพ็คเกจ (Package)



แพ็คเกจ (Package)

คำสั่ง package เป็นการระบุว่าคลาสอยู่ในแพ็คเกจใด โดยมีรูปแบบของคำสั่ง package ดังนี้

```
package <package_name> [<sub_package_name>] ;
```

ตัวอย่าง

```
package faculty.domain;
```

ซึ่งโปรแกรมภาษาจาวาหนึ่งโปรแกรมจะมีคำสั่ง package ได้เพียงคำสั่งเดียว โดยจะเป็นคำสั่งแรกของโปรแกรม และกรณีที่ไม่มีการสั่ง package คลาสจะถูกกำหนดไว้ในแพ็คเกจ default

คำสั่ง import

คำสั่ง import จะอยู่ก่อนหน้าการประกาศคลาส และหนึ่งโปรแกรมสามารถมีคำสั่ง import ได้หลายคำสั่ง

```
import <package_name>[.<sub_package_name>].<Class_name>
```

หรือ

```
import <package_name>[.<sub_package_name>].*;
```

ตัวอย่าง

```
import faculty.reports.Report;
```

หรือ

```
import java.awt.*;
```



THANKS !